

Repair the Amplifier, Not the Symptom: Stable World-Model Correction for Agent Rollouts

Xinyuan Song¹ Zekun Cai^{2,3}

¹Emory University, Atlanta, GA, USA ²The University of Tokyo, Tokyo, Japan

³LocationMind, Tokyo, Japan

xinyuan.song@emory.edu, caizekun@csis.u-tokyo.ac.jp

Abstract

As agent planning moves from short tool chains toward persistent workflows with thousands or tens of thousands of steps, failures will occur inside large planning graphs rather than in isolated predictions. Replanning the entire graph after every mistake is neither computationally realistic nor desirable: full-graph replay consumes large context budgets, exposes the LLM to many irrelevant symptoms, and can degrade long-context retrieval. This paper studies the missing component in such systems: a world-model corrector that repairs the failed planning graph in place. We compare two families of correctors. The first is the common engineering approach: scan nodes and edges, choose a suspicious local region, and ask an LLM to repair it. We implement strong engineering LLM correctors and find that they can help, especially when given very large contexts. The second family is our approach, **WM-SAR** (World-Model Subgraph Amplification Repair): instead of scanning for visible symptoms, it works backward from subgraph amplification, identifies the nodes and edges that keep re-amplifying error, and sends only that causal subgraph to the LLM. Across graph simulations and LLM repair experiments, WM-SAR substantially outperforms engineering correctors under realistic token budgets, achieves near-whole-graph stabilization with a compact region, and gives the LLM a cleaner repair target. Our code is available at: <https://github.com/Hik289/world-model-corrector.git>

1 Introduction

World models are becoming planning infrastructure. World models were introduced as learned simulators that let agents imagine future states before acting (Ha and Schmidhuber, 2018; Sutton, 1991). Modern variants support latent rollouts, search, model-based control, and relational state prediction (Chua et al., 2018; Hafner et al., 2019; Schrittwieser et al., 2020; Hafner et al., 2023;

Zhang et al., 2021; Feng et al., 2025). In parallel, LLM agents are moving from single-turn prediction toward tool use, web navigation, software engineering, and persistent exploration (Yao et al., 2023; Zhou et al., 2024b; Jimenez et al., 2024; Yang et al., 2024; Liu et al., 2024b; Shen et al., 2023; Wang et al., 2023). These two trends point to the same future system: an agent maintains a working world model, plans through it, updates it with tool feedback, and reuses it over many decisions.

Long planning naturally becomes a graph. In a short task, a rollout can be treated as a list of steps. In a long task, that view breaks down. A planner spawns tool calls; validators gate later actions; memory writes are reused by distant descendants; exceptions trigger recovery branches; final answers depend on several earlier sub-trajectories. The resulting object is a planning graph, not a flat transcript. As agents are deployed for longer workflows, this graph may contain thousands or tens of thousands of nodes. Replanning the whole graph after each error is therefore not a viable maintenance strategy. It consumes the context budget, forces the LLM to retrieve the relevant cause from a sea of irrelevant nodes, and is especially risky because long-context models can fail to use information placed away from the prompt boundaries (Liu et al., 2024a).

This creates the need for world-model correctors. A corrector is a maintenance robot for a planning graph. When a failure appears, the corrector should decide which part of the graph to revise, pass that region to an LLM or another repair module, and keep the rest of the world model intact. This is different from ordinary replanning: the corrector must identify a small causal region inside a much larger graph. If it chooses only the visible symptom, the next rollout can recreate the failure; if it chooses too much of the graph, the method becomes a costly full replay.

A strong default is an engineering LLM corrector. The most direct implementation is to scan the planning graph with engineering rules. One can rank nodes by error, take the top- K suspicious nodes, inspect high-error edges, choose a temporal window around the failure, expand a k -hop neighbourhood, run a cascade scan, or serialize a large region for an LLM root-cause prompt. This family is attractive because it is simple, model-agnostic, and compatible with feedback, self-refinement, and replanning methods (Yao et al., 2024; Madaan et al., 2023; Shinn et al., 2023; Zhou et al., 2024a; Cobbe et al., 2021; Lightman et al., 2023). We implement this engineering family carefully, including LLM-based versions. The result is a meaningful baseline: engineering correctors can work, and full-graph LLM repair can be strong on small graphs. Their limitation is the same limitation that motivates the problem. They spend budget by scanning visible evidence, so they may miss the small subgraph that re-amplifies error, and their best variant relies on contexts that become expensive and noisy as the planning graph grows.

Our corrector works backward from amplification. We propose **WM-SAR** (World-Model Subgraph Amplification Repair). Instead of scanning for the node that looks most wrong, WM-SAR asks which connected subgraph keeps the residual world model unstable. We represent a failed rollout as a typed graph and define a residual node-edge error operator. Its spectral radius determines whether remaining errors decay or amplify in the next rollout; proofs are in the appendix. WM-SAR uses this quantity to work backward from amplification: it scores local amplification, estimates node-edge coupling, grows a connected region by marginal spectral relief, and then sends only that region to an LLM. Figure 1 shows why engineering repair and amplification repair lead to different outcomes.

Contributions.

- **Problem formulation:** we formulate world-model correction as maintaining a failed planning graph under a limited repair budget, rather than re-planning the full graph.
- **Engineering correctors:** we implement a strong family of LLM and non-LLM engineering correctors based on node scans, windows, neighbourhoods, cascades, and full-graph prompting.

- **Amplification corrector:** we introduce WM-SAR, which infers the amplifying subgraph using GEAF, node-edge coupling, and marginal residual-spectral relief before invoking an LLM.
- **Theory and evidence:** we prove that residual spectral radius controls rollout error and planning regret, and show empirically that WM-SAR outperforms engineering correctors while using substantially smaller repair contexts.

2 Related Work

World models for planning. World-model planning traces back to architectures that interleave learning, planning, and acting (Sutton, 1991); modern learned world models simulate future states in latent space (Ha and Schmidhuber, 2018; Hafner et al., 2019, 2023) and can be coupled with search (Schrittwieser et al., 2020). Probabilistic dynamics models propagate uncertainty through imagined rollouts (Chua et al., 2018), while model-based policy optimization studies when short learned rollouts remain trustworthy under accumulated model bias (Janner et al., 2019). Asadi et al. (2018) establish Lipschitz continuity as a sufficient condition for bounded rollout error—a scalar criterion that our graph operator view lifts to structured agent plans. Graph-structured world models (Zhang et al., 2021; Feng et al., 2025; Battaglia et al., 2018; Hafner et al., 2019) extend this to relational state spaces, where message passing predicts how entities and dependencies evolve together. Anokhin et al. (2025) build episodic graph memories for agent planning. Recent LLM-agent systems also use world knowledge or environment dynamics to guide planning (Qiao et al., 2024; Chae et al., 2025). Our work addresses the previously unstudied problem of *how to repair* a failed GWM rollout: which subgraph to target and why.

LLM agents and failure repair. ReAct (Yao et al., 2023) interleaves reasoning and acting; Toolformer (Schick et al., 2023), Gorilla (Patil et al., 2024), RestGPT (Song et al., 2023), and ToolLLM (Qin et al., 2024) teach LLMs to invoke tools or APIs. Self-Refine (Madaan et al., 2023) applies iterative self-feedback to improve generation quality. Reflexion (Shinn et al., 2023) stores verbal feedback in memory, while LATS (Zhou et al., 2024a) combines language-agent planning with tree search. Verifier-based and process-supervised approaches provide more granular feedback for multi-step reasoning (Cobbe et al., 2021; Lightman et al., 2023).

Why Engineering Repair Fails to Suppress Error Amplification

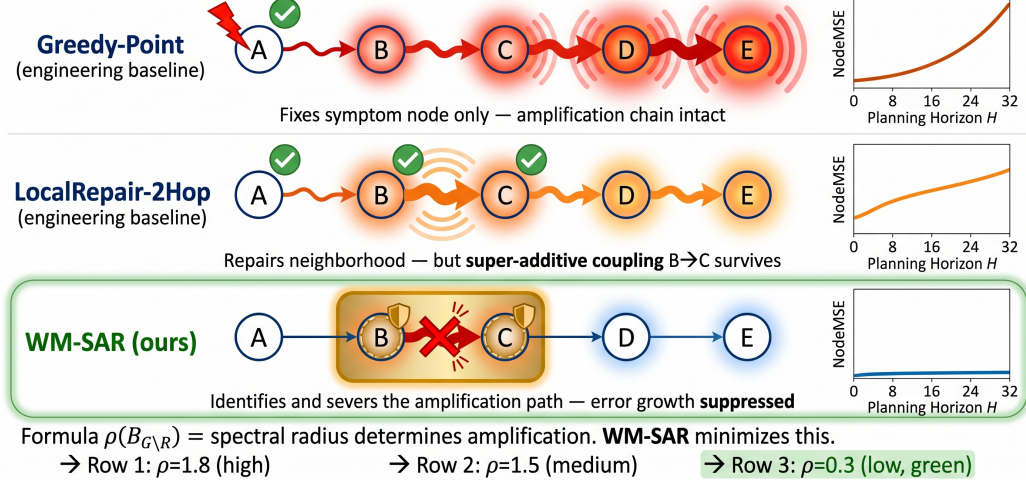


Figure 1: World-model correction should target the amplifier rather than the most visible symptom. Greedy repair fixes the highest-error node but leaves the causal path active. A fixed local neighbourhood can still miss the node-edge coupling that recreates the failure. WM-SAR selects the compact connected path whose removal lowers residual amplification, which flattens the empirical NodeMSE curve over horizon H .

Retroformer (Yao et al., 2024) uses policy gradient retrospection to correct failed ALFWorld (Shridhar et al., 2021) trajectories. AgentBench (Liu et al., 2024b) and AgentBoard (Ma et al., 2024) benchmark LLM agents across diverse tasks; TaskBench (Shen et al., 2023) evaluates tool-chaining task automation. SWE-bench (Jimenez et al., 2024) and SWE-agent (Yang et al., 2024) benchmark code repository repair; WebArena (Zhou et al., 2024b) benchmarks realistic web navigation agents. Section 7.7 simulates benchmark-inspired topologies to test whether WM-SAR depends on the synthetic calling-tree shape. Zhang et al. (2025) attribute task failures to specific agents in multi-agent LLM systems—a complementary problem to ours (attribution vs. repair). Lu et al. (2025) visualise LLM agent behaviour for interactive analysis. All these methods rely on linear trace scanning or full-trajectory replay; none uses graph spectral analysis to select a *minimal causal subgraph* for targeted LLM repair.

Graph-theoretic error analysis. Spectral graph theory (Chung, 1997) links graph eigenvalues to diffusion and propagation phenomena. Topping et al. (2022) study over-squashing in GNNs via Ricci curvature, showing that bottleneck edges cause information loss—our κ_v coupling factor identifies the analogous *amplification bottlenecks*. Gilmer et al. (2017) establish the message-passing neural network framework that underlies most

GWM architectures. Graph anomaly detection (Niu et al., 2023) and fault localisation (Lou et al., 2021) similarly seek to identify critical subgraphs, but in static graphs without a planning-horizon regret objective. Our GEAF (Graph Error Amplification Field) is the first per-node quantity derived from rollout error theory that enables *prospective* identification of amplification sources before invoking an LLM.

Subgraph selection and PageRank. PageRank (Page et al., 1999) scores nodes by global influence; TopK selection and k -hop expansion are standard heuristics in graph analysis. Gleave and Irving (2022) catalogue failure modes of LLM-based reward models, noting that local corrections can leave global pathology intact—consistent with our finding that node-level repair does not reduce $\rho(B)$. WM-SAR instead grows a *connected* region greedily under a $\Delta\rho$ -relief criterion, directly targeting the spectral quantity that controls the planning-regret bound.

3 Rollout Error Theory

We now state why a corrector should optimize amplification rather than visible error. In a large planning graph, the immediate symptom can be far downstream from the node or edge that will recreate the failure in the next rollout. A repair is therefore useful only if it lowers the operator that

propagates both node-state and edge-state errors. Proofs are deferred to the appendix.

Definition 1 (Node-state rollout error). *For a graph world model with true node states X_k and predicted node states $\hat{X}_k$ at rollout step k , define $e_k^X = \|\hat{X}_k - X_k\|_F$. Let ϵ_X denote the one-step model approximation error.*

Assumption 1 (Fixed-edge Lipschitz rollout). *For a fixed graph with adjacency matrix A , the message-passing update is globally Lipschitz in the node state with constant*

$$L_X = L_\sigma \rho(A) \prod_{\ell} \|W_\ell\|_2, \quad (1)$$

where L_σ is the activation Lipschitz constant and W_ℓ are the message-passing layer matrices.

Theorem 1 (Fixed-edge node-error growth). *Under Assumption 1, if $e_{k+1}^X \leq L_X e_k^X + \epsilon_X$, then for every $k \geq 1$,*

$$e_k^X \leq L_X^k e_0^X + \epsilon_X \frac{L_X^k - 1}{L_X - 1}. \quad (2)$$

For $L_X = 1$, the second term is $k\epsilon_X$.

The theorem separates two sources of risk: one-step approximation error and structural amplification. Engineering correctors mostly observe the first term through $e(v)$. WM-SAR also estimates the second term with $\widehat{\text{GEAF}}_v = e(v)\rho(A_v)w^H$, where $\rho(A_v)w^H$ is the local capacity to amplify the observed error.

Assumption 2 (Coupled node-edge channel). *Let $z_k = (e_k^X, e_k^A)^\top$ collect node-state and edge-state errors, and let $\varepsilon = (\epsilon_X, \epsilon_A)^\top$. Their one-step linearized upper envelope is*

$$z_{k+1} \preceq \mathbf{B} z_k + \varepsilon, \quad (3)$$

$$\mathbf{B} = \begin{pmatrix} L_X & L_A \\ M_X & M_A \end{pmatrix},$$

where all four entries are nonnegative. Here $\preceq$ denotes elementwise inequality.

Theorem 2 (Coupled node-edge amplification). *Under Assumption 2, the spectral radius of the coupled operator is*

$$\Delta_B = \sqrt{(L_X - M_A)^2 + 4L_A M_X},$$

$$\rho(\mathbf{B}) = \frac{L_X + M_A + \Delta_B}{2}. \quad (4)$$

If $L_A M_X > 0$, then $\rho(\mathbf{B}) > \max(L_X, M_A)$.

The local coupling score $\kappa_v = L_A(v)M_X(v)$ estimates how strongly node v participates in this super-additive node-edge channel. A node can therefore be dangerous even when its observed error is not the largest: it may sit on a node-edge channel that increases the Perron root of the residual operator.

Assumption 3 (Value sensitivity). *For any policy π , the reward is L_R -Lipschitz in rollout state, the policy selection map has sensitivity κ , and the reward approximation error per step is bounded by ϵ_R . The coupled model error satisfies $\|z_k\| \leq \epsilon \rho(\mathbf{B})^k$ for $z_k = (e_k^X, e_k^A)$.*

Theorem 3 (Planning-regret bound). *Let π^* be the optimal policy under the true rollout dynamics and $\hat{\pi}$ the policy selected under the learned GWM. Under Assumption 3,*

$$J(\pi^*) - J(\hat{\pi}) \leq 2L_R \kappa \epsilon \Phi_H(\gamma, \rho(\mathbf{B})) + 2\epsilon_R H, \quad (5)$$

where

$$\Phi_H(\gamma, \rho) = \sum_{t=0}^{H-1} (\gamma \rho)^t. \quad (6)$$

The bound makes the repair criterion explicit. For a repaired region R , the future rollout is controlled by the residual spectral radius $\rho(\mathbf{B}_{G_f \setminus R})$. A scanning corrector can repair high-error nodes while leaving this quantity large. WM-SAR instead chooses the connected subgraph whose correction lowers the residual operator, which is why it can outperform engineering correctors even when they see comparable local evidence.

4 Problem Formulation

The repair problem is observable after a failure: we have the failed trace, typed dependencies, node-level error estimates, and a budget for re-running or revising part of the trace. The future deployment constraint is that this budget is small relative to the full planning graph. A corrector cannot assume that it can serialize thousands of nodes into an LLM prompt after every failure. The target is therefore not the largest error node, but the smallest connected region whose correction makes the remaining world model stable.

Definition 2 (Failure graph). *A failed rollout τ is represented as $G_f = (V, E, \mathbf{X}, t^*)$, where nodes are agent calls and edges are causal dependencies. Each node $v \in V$ carries an observed error magnitude $e(v)$, uncertainty $u(v)$, repair cost $c(v)$, and*

a node type (planner, executor, validator, checker, aggregator, reporter, logger, error_handler, or final_answer). Edge types encode calls, validation, reporting, error routing, triggering, and logging. The sink t^* is the final-answer node, and $\mathcal{R}^*$ denotes the ground-truth corrupted region used only for evaluation.

Definition 3 (Repair objective). Given budget $K_{\max}$, the spectral repair region is

$$R^* = \arg \min_{R \subseteq V: G_f[R] \text{ connected}, |R| \leq K_{\max}} \rho(\mathbf{B}_{G_f \setminus R}). \quad (7)$$

Exactly minimizing $\rho(\mathbf{B}_{G_f \setminus R})$ over connected subsets with bounded size is NP-hard in general because it contains minimum spectral cut as a special case. WM-SAR is the greedy approximation used in our experiments.

Engineering baselines instead optimize variants of $\sum_{v \in R} e(v)$. This is a useful attribution signal, but it is not a stability objective: removing a large downstream symptom may leave the residual Peron root unchanged.

5 WM-SAR Algorithm

We study two kinds of world-model correctors. An *engineering corrector* scans the planning graph directly: it chooses the largest-error node, a TopK set, a temporal window, a local neighbourhood, or a large serialized trace for an LLM. This is the default design pattern because it is easy to implement and often works when the graph is small. WM-SAR is an *amplification-based corrector*. It first estimates which subgraph keeps error alive in the residual world model, then asks the LLM to repair only that subgraph. Figure 2 summarizes this division. The graph phase uses the spectral-graph principle that eigenvalues govern propagation (Chung, 1997); the LLM phase uses structured tool-context prompting (Yao et al., 2023; Schick et al., 2023; Patil et al., 2024; Song et al., 2023; Qin et al., 2024). The goal is not to avoid LLM reasoning, but to give the LLM the right graph slice before its context budget is spent.

5.1 Phase 1: Finding the Repair Region

Step 1 - Per-node GEAF score. Engineering correctors begin with observed error. WM-SAR begins with observed error multiplied by the local ability to amplify it. The fixed-edge theorem shows that rollout error grows with the local Lipschitz-

amplification scale, so we use $\widehat{\text{GEAF}}_v$ as a graph-only proxy:

$$\widehat{\text{GEAF}}_v = e(v) \rho(A_v) w^H, \quad (8)$$

where $e(v)$ is the observed error at v ; A_v is the $n_v \times n_v$ adjacency matrix of v ’s H -hop subgraph G_v ; $\rho(A_v)$ is its spectral radius (how much v ’s neighbourhood amplifies signals); and $w \in (0, 1)$ is a horizon discount factor. All default hyperparameters are listed in Appendix C. We use this spectral-radius proxy rather than a walk-count alternative ($\sum_{k=1}^H (A^k \mathbf{1})_v$); both are monotonic in structural amplification. This is analogous in spirit to influence scores such as PageRank, but it is local to a repair candidate and tied to rollout stability (Page et al., 1999; Chung, 1997).

Intuitively, $\rho(A_v)$ measures *structural* amplification—how many independent paths carry error out of v —while $e(v)$ measures the *observed* error magnitude. A node with high $e(v)$ but low $\rho(A_v)$ is a dead-end symptom node; a node with low $e(v)$ but high $\rho(A_v)$ is a latent amplifier. WM-SAR targets nodes where both are high.

Step 2 - Coupling factor. From the coupled node-edge theorem, the **coupling factor** $\kappa_v = L_A(v) \cdot M_X(v)$ captures the super-additive interaction between v ’s input and output channels:

$$\begin{aligned} L_A(v) &= w \beta_A \bar{d}_{\text{in}}(v) \cdot \bar{e}(v), \\ M_X(v) &= w \beta_X \bar{d}_{\text{out}}(v) \cdot \bar{e}(v), \end{aligned} \quad (9)$$

where $\bar{d}_{\text{in}}(v) / \bar{d}_{\text{out}}(v)$ are the average in/out edge-type diversity of v , $\bar{e}(v)$ is the mean error in v ’s neighbourhood, and β_A, β_X are calibration constants described in Appendix C. Nodes with high κ_v lie on *cross-coupling paths*: repairing them drives down $\rho(\mathbf{B})$ through both state and edge channels.

Step 3 - Seed selection. We rank every node by a combined seed score:

$$\text{score}_{\text{seed}}(v) = e(v) \widehat{\text{GEAF}}_v (1 + \kappa_v). \quad (10)$$

and select the top- k_0 nodes as candidate seeds. Each seed s initiates an independent growing process.

Step 4 - $\rho(\mathbf{B})$ -relief growing. This is where WM-SAR departs most sharply from engineering scanning. Starting from $R = \{s\}$, it greedily

WM-SAR Pipeline

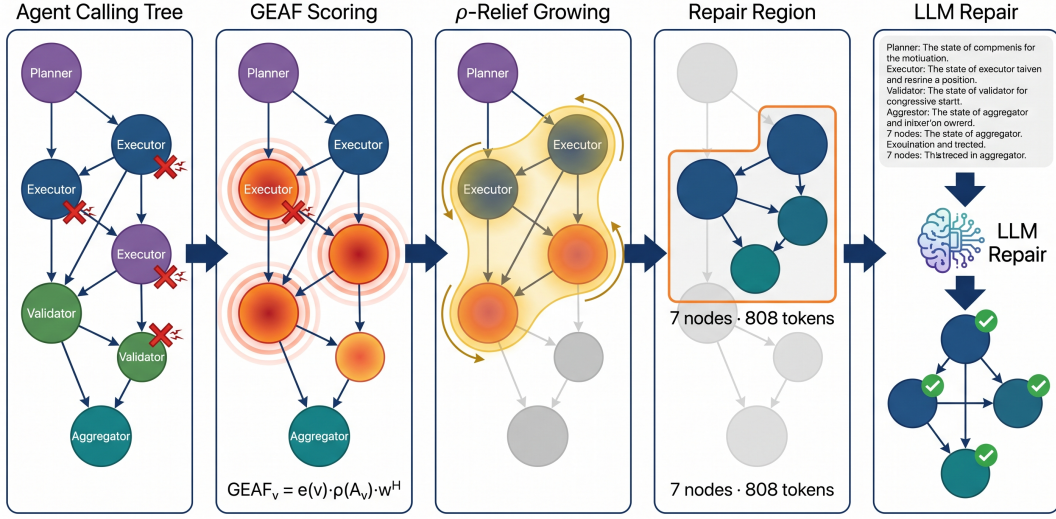


Figure 2: WM-SAR workflow. Region selection first scores every node by local amplification and node-edge coupling, seeds compact candidate regions, expands them by marginal residual-spectral relief, and prunes redundant nodes. Only the final connected region is serialized for LLM root-cause repair, keeping the language-model context focused on the amplifying subgraph.

expands R by adding the boundary node u^* that maximally reduces the residual spectral coupling:

$$u^* = \arg \max_{u \in \partial R} g_R(u), \quad (11)$$

$$g_R(u) = \lambda_1 e(u)(1 + \kappa_u) + \lambda_2 \Delta \rho_{\text{rel}}(u) - \lambda_3 c(u).$$

where ∂R is the frontier of nodes adjacent to R in G_f ;

$$\Delta \rho_{\text{rel}}(u) = \rho(\mathbf{B}_{G_f \setminus R}) - \rho(\mathbf{B}_{G_f \setminus (R \cup \{u\})}), \quad (12)$$

$$c(u) = 1 + \frac{|N(u) \cap R|}{|R|}. \quad (13)$$

Here $\Delta \rho_{\text{rel}}(u)$ is the marginal $\rho(\mathbf{B})$ -reduction gained by adding u , and $c(u)$ penalises redundant already-covered nodes. $\lambda_1, \lambda_2, \lambda_3$ balance error coverage, spectral relief, and cost penalty (values in Appendix C). Growing stops when no node provides positive gain or the budget K_{max} is reached.

Why this beats engineering baselines. TopK selects high-error nodes independently, so the chosen nodes can be disconnected symptoms. Window repair assumes temporal adjacency, so it can miss a root cause that is far upstream. LocalRepair expands by graph distance, so it may spend budget on harmless neighbours while missing a high-gain coupling edge. Full-graph LLM repair can recover the answer on small graphs, but its cost grows with the whole planning graph and long contexts are not reliably used by current LLMs (Liu et al., 2024a).

WM-SAR instead uses $\Delta \rho_{\text{rel}}$ to ask how much each candidate node or edge lowers the residual amplifier, the same quantity that appears in the planning-regret bound. This makes the corrector closer to fault-localization and critical-subgraph methods (Lou et al., 2021; Niu et al., 2023) than to generic trace scanning.

Step 5 - Pruning and selection. After growing, we prune: remove any $v \in R$ such that $\rho(\mathbf{B}_{G_f \setminus (R \setminus \{v\})}) \leq \rho(\mathbf{B}_{G_f \setminus R})$ (removing v does not increase residual amplification). Among all k_0 candidate regions, we return the one maximising

$$\text{Score}(R) = \frac{(\sum_{v \in R} e(v)(1 + \kappa_v)) \Delta \rho_{\text{rel}}(R)}{1 + |R|}. \quad (14)$$

This score is deliberately residual: it rewards regions that lower $\rho(\mathbf{B}_{G_f \setminus R})$, not regions that merely contain high-error nodes. Uniform uncertainty in this dataset carries no ranking signal, so the uncertainty term is inactive at the default hyperparameters; the ablation in Table 2 verifies that removing it has no effect.

The full procedure is summarised in Algorithm 1.

5.2 Phase 2: LLM-Based Repair

Once WM-SAR selects R^* , the LLM stage is deliberately simple. We serialize only the selected subgraph, preserving node types, local state

Algorithm 1 WM-SAR Region Selection (greedy approximation to Definition 3)

Require: Failure graph G_f , budget $K_{\max}$, parameters $(H, w, k_0, \lambda_1, \lambda_2, \lambda_3)$

Ensure: Connected repair region R^*

```

1: Compute  $\widehat{\text{GEAF}}_v \leftarrow e(v) \cdot \rho(A_v) \cdot w^H$  and
    $\kappa_v \leftarrow L_A(v) \cdot M_X(v)$  for every  $v \in G_f$ 
2:  $\mathcal{S} \leftarrow$  top- $k_0$  nodes by  $e(v) \cdot \widehat{\text{GEAF}}_v \cdot (1 + \kappa_v)$ 
3: for each seed  $s \in \mathcal{S}$  do
4:    $R \leftarrow \{s\}$ 
5:   while  $|R| < K_{\max}$  do
6:      $u^* \leftarrow \arg \max_{u \in \partial R} \lambda_1 e(u)(1 + \kappa_u) +$ 
        $\lambda_2 \Delta \rho_{\text{rel}}(u) - \lambda_3 c(u)$ 
7:     if  $\Delta \rho_{\text{rel}}(u^*) > 0$  then
8:        $R \leftarrow R \cup \{u^*\}$ 
9:     else break
10:    end if
11:  end while
12:  Remove  $v$  from  $R$  if  $\rho(\mathbf{B}_{G_f \setminus (R \setminus \{v\})}) \leq$ 
     $\rho(\mathbf{B}_{G_f \setminus R})$ 
13: end for
14: return  $R^* \leftarrow \arg \max_R \text{Score}(R)$  where
     $\text{Score}(R) = \frac{\sum_v e(v)(1 + \kappa_v) \cdot \Delta \rho_{\text{rel}}(R)}{1 + |R|}$ 

```

features, and incoming/outgoing boundary edges. The LLM is then asked to identify the root-cause node(s) and produce a structured repair decision. In our simulated evaluation, the repair operator zeroes the selected node errors and re-runs the rollout; in a deployed agent, the same decision would trigger a targeted re-invocation of the corresponding planner, tool call, validator, or memory update. Prompt and serialization details are in Appendix E.

This stage is where token efficiency matters. A full-graph LLM corrector can be competitive on small graphs, but it pays for every irrelevant node and asks the model to retrieve the causal path from a long context. WM-SAR instead spends the LLM budget on the subgraph most likely to change the next rollout. Figure 4 and Table 4 report the resulting token–quality tradeoff.

6 Dataset: Agent Calling-Tree Testbed

We construct a synthetic heterogeneous calling-tree testbed that mimics the typed call structure of planner-executor-validator agents.

Graph structure. Each instance is a medium-size typed calling tree with agent roles such as planner, executor, validator, checker, and final-answer

nodes. Edges encode calls, validation, reporting, error routing, triggering, and logging. Each node stores the state features needed to distinguish a root error from a downstream cascade victim; full statistics are in Appendix D.

Failure injection. Failures originate at an upstream planner or executor and then propagate along the typed dependency graph. This creates the intended ambiguity: the largest observed error is often downstream of the node that introduced the mistake. The ground-truth corrupted region is used only for evaluation.

Multi-step error metrics. After zeroing errors in R (simulated repair), remaining errors propagate via the fixed-edge recursion in Equation 2: $e_{k+1}(v) = L_X \cdot e_k(v) + L_A \sum_{u \rightarrow v} e_k(u)$. We report long-horizon NodeMSE, GrowthSlope, and residual spectral-radius reduction; these metrics separate immediate error removal from true rollout stabilization.

7 Experiments

7.1 Baselines: Engineering Correctors

All baselines use the same simulated repair operator: selected node errors are zeroed and the rollout is re-simulated. The only variable is the corrector’s choice of repair region. This isolates the central question: does success come from engineering scan rules, or from selecting the subgraph that actually controls residual amplification?

- **Greedy-Point(K=1):** top-1 by error.
- **TopK-Point(K=3/5):** top- K by error (may be disconnected).
- **Window-2/4/8-Point:** sliding window of k steps; best-mean-error window.
- **LocalRepair-2/3Hop:** k -hop neighbourhood of the top-error node (the larger neighbourhood is close to whole-graph repair in this testbed).
- **CascadeRepair:** topological scan until error drops below threshold.
- **Oracle:** exact GT region $\mathcal{R}^*$.
- **LLM engineering correctors:** serialize regions selected by the above rules, including full-graph variants, and ask the same LLM to identify the root cause.
- **WM-SAR:** amplification-based corrector using Definition 3 and Algorithm 1.

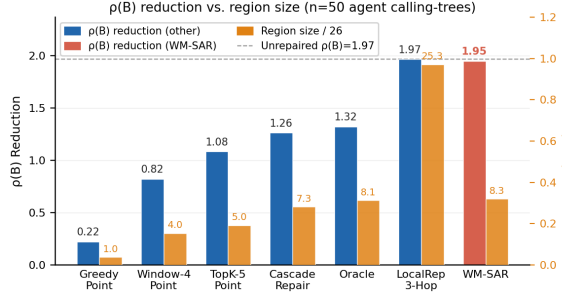


Figure 3: Spectral relief per repaired region. Engineering correctors can reduce error, but many spend budget on nodes that do little to lower the residual amplifier. WM-SAR reaches the high-relief regime with a small connected subgraph, whereas fixed-hop repair obtains similar relief mainly by expanding toward whole-graph replay.

7.2 Main Results

Table 1 gives the main comparison. The engineering correctors are not weak strawmen: neighbourhood and cascade repair often find plausible corrupted regions, and large LLM contexts improve root-cause recovery. The failure is more specific. They do not directly optimize the residual amplifier, so under a compact repair budget they often leave a graph that will recreate the same error.

Finding 1: engineering scans repair symptoms but often leave the amplifier. Greedy, TopK, and window correctors repair nodes that look bad in the observed trace, and this can reduce short-horizon error. Figure 3 shows why the improvement does not last: these methods leave most residual spectral mass in place. The symptom has been patched, but the causal channel that recreates the symptom is still present.

Finding 2: amplification repair turns correction into stabilization. WM-SAR selects about eight nodes on average, but those nodes are the ones that control the residual operator. After they are repaired, the rollout no longer behaves like a delayed failure: the error curve flattens, matching the stable regime predicted by the fixed-edge theorem. The horizon curve in Figure 6 makes this distinction visible: WM-SAR does not merely reduce the first repaired step; it prevents error from re-growing.

Finding 3: WM-SAR gets the benefit of brute force without brute force. LocalRepair-3Hop succeeds mainly because this testbed is small enough that a three-hop neighbourhood almost covers the whole graph. WM-SAR reaches the

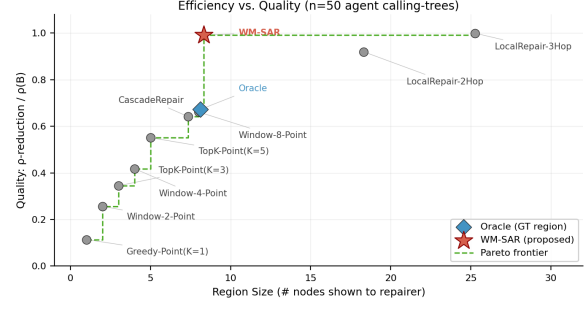


Figure 4: LLM repair tradeoff after graph-based context selection. Each point is a corrector whose selected region is serialized for the same LLM root-cause prompt; point size indicates token cost. Full-graph engineering repair can be competitive but is token-heavy. WM-SAR sits on the efficient frontier because it preserves the causal bridge while omitting downstream symptoms.

same spectral regime with roughly $3\times$ fewer nodes, which is the behavior we want in larger agent traces where full replay is impractical. The budget sweep in Figure 7 shows the same effect under fixed region-size constraints.

Finding 4: overlap is not the same as repair. CascadeRepair often overlaps the corrupted region, but it can still leave the amplifying channel active. This separates attribution from repair: a method can identify many corrupted nodes and still fail to change the dynamics that will govern the next rollout. The coupling term in WM-SAR is what turns region selection toward stability rather than coverage alone. Table 2 confirms that growing the connected region is the decisive component, while Table 3 shows that the same ordering holds on benchmark-inspired code, web, and operating-system agent topologies.

7.3 LLM Repair: Single-API + Cross-Model Study

We use the LLM for actual root-cause prediction. For each instance, each method shows a different subset of the failure graph to the LLM in structured natural language and asks it to identify the root-cause node. The main evaluation uses GPT-4o-mini across all region-selection methods; a cross-model study in Figure 5 tests whether the same context-selection advantage appears with GPT-4o, GPT-4o-mini, and Gemini-2.5-Flash (OpenAI, 2024a,b; Google Cloud, 2026). The benchmark-inspired graph families follow SWE-bench, WebArena, and AgentBench-style agent workflows (Jimenez et al., 2024; Zhou et al., 2024b; Liu et al., 2024b; Yang

Table 1: Main repair comparison on the agent calling-tree testbed. Every method uses the same state-correction operator, so differences come only from the corrector’s region choice. Engineering correctors repair visible symptoms or broad local neighbourhoods; WM-SAR repairs the subgraph that lowers residual amplification. It reaches near-whole-neighbourhood stabilization with a compact region and keeps the post-repair slope close to flat. The Oracle repairs the injected corrupted region; it is an attribution upper bound rather than a spectral-relief optimizer.

Method	Size↓	Conn%↑	IoU↑	$\rho(\mathbf{B})$ -red↑	MSE@4↓	MSE@16↓	MSE@32↓	Slope↓
<i>Pointwise methods</i>								
Greedy-Point(K=1)	1.0	100	.144	$0.22 \pm .06$	99.0	184.7	185.2 ± 18	$+.005$
TopK-Point(K=3)	3.0	36	.433	$0.68 \pm .09$	69.3	143.3	144.1 ± 15	$+.009$
TopK-Point(K=5)	5.0	48	.630	$1.08 \pm .10$	46.6	107.3	107.6 ± 14	$+.007$
<i>Window-based methods (sliding context window)</i>								
Window-2-Point	2.0	30	.251	$0.50 \pm .08$	91.2	184.8	185.5 ± 18	$+.009$
Window-4-Point	4.0	12	.317	$0.82 \pm .09$	80.0	182.9	183.8 ± 17	$+.011$
Window-8-Point	8.0	22	.360	$1.30 \pm .09$	61.2	169.3	170.8 ± 16	$+.019$
<i>Neighbourhood / cascade methods</i>								
LocalRepair-2Hop	18.3	100	.362	$1.81 \pm .06$	27.5	91.8	92.8 ± 16	$+.009$
LocalRepair-3Hop	25.3	100	.325	$1.97 \pm .05$	0.2	3.3	3.3 ± 1.7	$+.001$
CascadeRepair	7.3	96	.871	$1.26 \pm .10$	4.9	57.6	61.1 ± 10	$+.036$
WM-SAR (ours)★	8.3	94	.845	$1.95 \pm .05$	6.0	6.3	6.3 ± 1.1	$+.0004$
Oracle†	8.1	100	1.00	$1.32 \pm .10$	0.0	0.0	0.0	$-.013$

Size = mean region size. Conn = % connected instances. $\rho(\mathbf{B})$ -red = $\rho(\mathbf{B}_{G_f}) - \rho(\mathbf{B}_{G_f \setminus R})$. Slope = GrowthSlope $\times 10^0$.

et al., 2024). The complete single-API table appears in Table 4.

WM-SAR gives the LLM the right context. The LLM experiment asks the practical question faced by a corrector robot: if the downstream repair model is fixed, which part of a large planning graph should it see? WM-SAR gives the strongest exact root-cause recovery while using a compact context. Full-graph prompts contain the answer somewhere, but they also contain many irrelevant downstream symptoms and scale poorly with planning-graph size; pointwise prompts are cheap, but they often omit the causal bridge. The full single-API comparison appears below in Table 4; it shows that WM-SAR improves exact recovery while using less than half the tokens of full-graph LLM repair.

The advantage is not tied to one API. Across the three LLM tiers in Figure 5, the same ordering appears: WM-SAR is best on every model, while LocalRepair and TopK move with model strength but do not close the context-selection gap. The gain is therefore not a quirk of one API; it comes from showing the model a cleaner causal subgraph. WM-SAR improves the corrector’s input, not the LLM itself.

The remaining experiment subsections bring the secondary checks into the main text: horizon stability, ablation, benchmark-topology transfer, token-budget efficiency, cascade-gain robustness, full

LLM repair, attribution boundaries, and the full baseline table.

Spectral relief explains the method classes.

Across methods, the qualitative ordering follows the theory: methods that reduce residual spectral radius also suppress long-horizon error. The relationship is class-level rather than instance-level; the spectral objective does not predict every graph perfectly, but it explains why compact amplification repair behaves differently from pointwise repair.

7.4 Horizon Stability

The main table reports fixed horizons; Figure 6 asks a harsher question: after a local repair, does error stay low as the rollout grows? This is where symptom repair separates from amplifier repair. Pointwise and window correctors lower the first few errors but still diverge at long horizons. LocalRepair-3Hop is stable because it nearly covers the whole graph. WM-SAR reaches the same stable regime with a compact region.

7.5 Multi-Step Error Profile

Figure 6 plots NodeMSE as a function of horizon H ($n = 50$, seed=42). Pointwise/window methods cannot flatten the error curve; LocalRepair-3Hop drives it to near-zero by covering $\approx 99\%$ of the graph; WM-SAR achieves near-zero MSE with only 8.3 nodes.

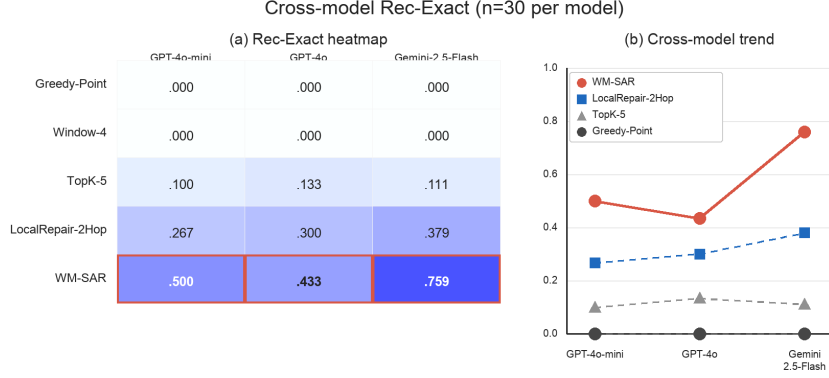


Figure 5: Cross-model LLM root-cause identification under the same repair prompt. The heatmap reports Rec-Exact for each method-model pair, and the trend plot compares WM-SAR against engineering correctors across GPT-4o-mini, GPT-4o, and Gemini-2.5-Flash. The gain persists across model tiers, which indicates that the selector improves the evidence shown to the LLM rather than exploiting one particular API.

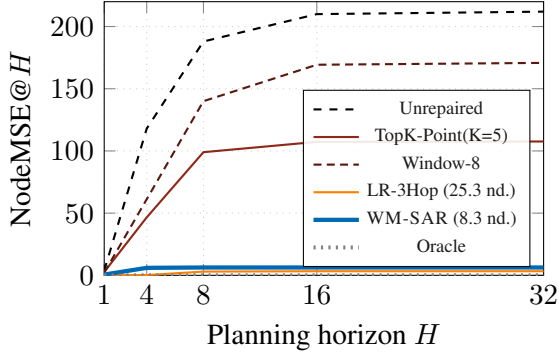


Figure 6: NodeMSE@ H ($n = 50$, seed=42). Point-wise/window methods diverge. LR-3Hop flattens by covering $\approx 99\%$ of the graph. WM-SAR achieves near-zero MSE with only 8.3 nodes.

7.6 Ablation Study

Table 2 isolates which parts of WM-SAR are needed for spectral repair. The growing phase is the decisive component: without it, a high-scoring seed remains a point repair and does not remove the amplifying path.

Removing the Growing phase (seed-only) collapses recovery: $\rho(\mathbf{B})$ -reduction 0.65, MSE@32 returns to ≈ 210 (near-unrepaired), GrowthSlope reverts to $+0.0026$. Removing ρ -relief (use error-only growing) reduces $\rho(\mathbf{B})$ -reduction by $\sim 33\%$ but does not change MSE@32 or slope in this small-graph regime, since error-greedy growing selects similar nodes. GEAF and coupling provide equivalent information on the synthetic dataset. Pruning keeps quality intact but shrinks the region from 13.5 to 8.3 nodes (-40%).

Table 2: Ablation on the calling-tree testbed ($n = 50$, seed=42). ρ -red denotes residual spectral-radius reduction; Slp denotes GrowthSlope; Sz denotes mean region size. Removing growing collapses WM-SAR back to a seed-only point repair.

Config	ρ -red	MSE@32	Slp	Sz
Full WM-SAR	1.95	6.3	.0004	8.3
—GEAF (error seed)	1.95	6.3	.0004	8.3
—Coupling	1.95	6.3	.0004	8.3
— ρ -relief	1.30	6.3	.0004	7.6
—Pruning	1.95	6.3	.0004	13.5
— Growing	0.65	210	.017	1.0
—Uncertainty*	1.95	6.3	.0004	8.3

* Uncertainty score is computed but inactive at default hyperparameters; removing it has no effect on spectral metrics.

7.7 Benchmark Topology Generalisation

To validate that WM-SAR generalises beyond the synthetic calling-tree testbed, we simulate three benchmark-inspired topology simulations ($n = 50$ each, seed=42)—synthetic graphs matching the reported topologies of each benchmark (not runs of the actual benchmark systems)—modelling node types, edge structures, and failure cascades documented in each paper (Jimenez et al., 2024; Zhou et al., 2024b; Liu et al., 2024b):

- **SWE-bench** (Jimenez et al., 2024): $N \approx 21$ nodes. IssueAnalyzer $\rightarrow$ FileLocator $\times 4 \rightarrow$ CodeAnalyzer $\rightarrow$ PatchWriter $\rightarrow$ TestRunner. Cascade gain $\alpha = 1.15$.
- **WebArena** (Zhou et al., 2024b): $N \approx 15$ nodes. TaskPlanner $\rightarrow$ Navigator chain $\rightarrow$ PageReader $\rightarrow$ FormFiller $\rightarrow$ Submitter. Cascade gain $\alpha =$

1.08.

- **AgentBench-OS** (Liu et al., 2024b): $N \approx 12$ nodes. Commander $\rightarrow$ BashNode chain $\rightarrow$ OutputParser $\rightarrow$ Verifier. Cascade gain $\alpha = 1.12$.

Table 3 shows that the spectral advantage is not tied to a single synthetic topology: WM-SAR is best or tied-best on all three benchmark-inspired graph families.

Table 3: Benchmark-topology generalisation ($n = 50$ per topology, seed=42). The simulations match the reported causal structure of SWE-bench, WebArena, and AgentBench-OS rather than using real benchmark traces. ρ -r denotes $\rho(\mathbf{B})$ -reduction and Sz denotes mean region size.

Method	SWE-bench $\rho_0=1.92$		WebArena $\rho_0=1.24$		AgentBench $\rho_0=1.39$	
	ρ -r	Sz	ρ -r	Sz	ρ -r	Sz
Greedy-Pt	1.43	1	0.59	1	0.88	1
Window-4	1.44	4	0.89	4	1.08	4
TopK-5	1.45	5	0.84	5	1.06	5
LR-2H	1.65	18	0.47	8	1.26	11
LR-3H	1.92	21	1.14	11	1.35	12
Cascade	1.55	15	1.08	13	1.39	12
WM-SAR	1.79	17	1.24	14	1.39	11

WM-SAR is the best non-brute method on every benchmark. On SWE-bench the high branching ($K = 4$ file locators) creates strong cross-coupling that pointwise methods cannot reach. On WebArena’s linear chain LR-2Hop covers $\approx 50\%$ of nodes yet still under-performs WM-SAR, which targets the Navigator $\rightarrow$ PageReader coupling with ≈ 14 nodes.

7.8 Budget Efficiency Analysis

Figure 7 varies the explicit region-size budget and shows that WM-SAR reaches high spectral relief before engineering methods have enough budget to cover the relevant amplification path.

WM-SAR is highly budget-efficient. At $K = 3$, WM-SAR attains ρ -reduction 1.45, exceeding the Oracle-region baseline in this sweep (1.42; $\approx 102\%$). Engineering methods at $K = 3$ reach only 0.62–0.77. WM-SAR’s region grows from a κ -seeded core and saturates around $K = 8$ because each added node must satisfy $\Delta\rho_{\text{rel}} > 0$ by construction (Equation 11); engineering methods admit downstream symptom nodes that do not lie on the amplification path and therefore contribute nothing to $\rho(\mathbf{B})$ relief.

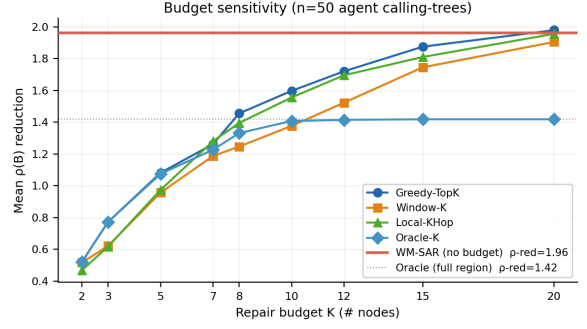


Figure 7: Budget sensitivity: $\rho(\mathbf{B})$ -reduction versus region size K ($n = 50$, seed=42). The red horizontal line is WM-SAR’s selected region (not forced to a fixed K), which reaches ρ -reduction 1.96. Engineering methods need $K \geq 15$ to approach this value. The grey dotted line is the Oracle-region baseline in this budget sweep.

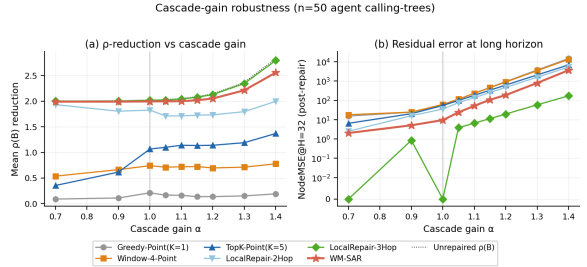


Figure 8: Cascade-gain sensitivity ($n = 50$, seed=42, $\alpha \in [0.7, 1.4]$). Left: $\rho(\mathbf{B})$ -reduction grows monotonically with α for every method; WM-SAR tracks LR-3Hop (full-graph) while Greedy stays near-zero. Right: post-repair NodeMSE@32 stays ≤ 10 for WM-SAR across the entire α range while engineering baselines diverge.

7.9 Cascade Gain Robustness

We vary the cascade gain $\alpha \in [0.7, 1.4]$ while holding the graph topology fixed (Figure 8). Pre-repair $\rho(\mathbf{B})$ rises from 2.00 to 2.83 as α grows; Coupled-operator super-additivity holds in 100% of instances for every α . WM-SAR maintains $\geq 90\%$ repair efficiency ($\rho(\mathbf{B})$ -reduction / pre-repair $\rho(\mathbf{B})$) across all α ($\geq 95\%$ within $\alpha \in [0.7, 1.2]$; 90.3% at $\alpha = 1.4$), matching LR-3Hop at low gains and staying within $\sim 10\%$ at $\alpha = 1.4$; Greedy-Point collapses to $< 10\%$ at all gains. Our main-experiment setting ($\alpha = 1.1$) sits in the middle of this range.

7.10 Full LLM and Baseline Evidence

The preceding LLM figure shows the cross-model trend. We now report the complete single-API and full-baseline tables in the main text because they

settle the practical question: WM-SAR is more accurate at the context scale a real corrector can afford.

7.11 LLM Repair: Full 11-Method Table

Table 4 and Figure 9 give the complete single-API view behind the main text. The result is budget efficiency: WM-SAR improves exact root-cause recovery without relying on a larger context, using less than half the tokens of Full-Graph-LLM.

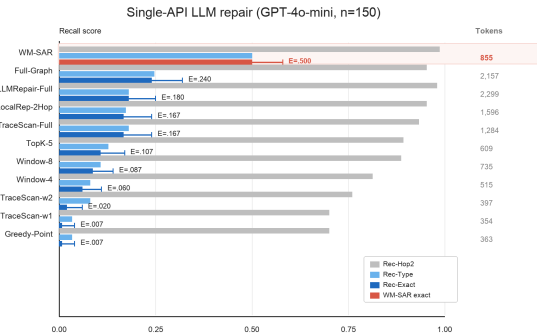


Figure 9: Single-API LLM root-cause repair (GPT-4o-mini, $n = 150$). Bars show Rec-Hop2, Rec-Type, and Rec-Exact; horizontal whiskers are 95% intervals for Rec-Exact. Token counts are shown at right. WM-SAR obtains the highest Rec-Exact while using a compact 855-token region.

7.12 Cross-Model Study: 3 LLMs

Table 5 reports the 3-LLM cross-model study ($n = 30$ instances per model, seed=42). WM-SAR achieves the highest Rec-Exact on all three model tiers.

7.13 Applicability Boundary: Who&When Attribution Benchmark

Table 6 reports results on the Who&When multi-agent failure attribution benchmark (Zhang et al., 2025) ($n = 30$ algorithm-generated traces, seed=42). This is the **applicability boundary** for WM-SAR: without per-node error signals, GEAF degenerates and spectral seeding loses its structural advantage, as discussed in the main text.

WM-SAR achieves 0.20 Rec-Exact vs. LocalRepair-2Hop’s 0.60 (−40 pp) because the Who&When dataset does not expose per-node intermediate error magnitudes; without $e(v)$, GEAF scoring defaults to the error sink node, matching LastError-Point’s behaviour. The graceful degradation to Rec-Agent=0.47 (above LastError-Point 0.30) shows that WM-SAR still

captures some agent-level causal structure; but step-level attribution requires intermediate error exposure.

7.14 Full Baseline Comparison

Table 7 gives the full engineering comparison on $n = 200$ instances. The baselines match the repairs a practical LLM corrector would try before adding a spectral objective. TraceScan methods linearize the execution trace and select either the whole trace, a fixed window, the first failed call, or the last erroneous call; these are standard debugging and failure-attribution heuristics for agent traces (Zhang et al., 2025; Lu et al., 2025). Top-B-Nodes and Top-B-Edges are budgeted attribution baselines that select the largest observed node or edge errors, analogous to saliency-style top- K repair. k-hop-k2/LocalRepair-2Hop is the graph-neighbourhood repair used in fault localization and subgraph debugging (Lou et al., 2021; Niu et al., 2023). PageRank-Subgraph replaces observed error with global graph influence (Page et al., 1999). Uncertainty-Subgraph selects high-uncertainty nodes, following the common uncertainty triage rule in model-based repair. LLMRepair-Full-Plan serializes the whole planning graph and asks the same LLM repair prompt used by WM-SAR; it is the full-context version of LLM self-repair and reflection-style baselines (Shinn et al., 2023; Madaan et al., 2023; Zhou et al., 2024a). Because long contexts are costly and can be hard for LLMs to use reliably (Liu et al., 2024a), this full-plan method is an upper-cost comparator rather than a scalable solution.

The table uses the original experimental names. Recovery is the fraction of instances where the root-cause region is identified and zeroed; Tok/Rec is mean tokens per successful recovery; PD-red is propagation-depth reduction in hops; DE-red is downstream-error reduction.

WM-SAR matches Oracle-Region and LLMRepair-Full-Plan on Recovery (0.625), but its token cost is much smaller: 1,389 tokens per successful recovery versus 10,333 for LLMRepair-Full-Plan. Pointwise trace scanners (TraceScan-w1/w2, LastError, FirstFailedCall) fail because the root cause often lies outside the selected window. TraceScan-Full-Point can approach WM-SAR’s recovery only by paying about $15\times$ the token cost. The pattern is the same as in Table 1: engineering correctors can find evidence, but efficient repair requires selecting the

Table 4: LLM root-cause identification (GPT-4o-mini, $n = 150 = 50 \text{ instances} \times 3 \text{ seeds}$). Rec-E/T/H = Rec-Exact/Type/Hop2. Tok = mean tokens. Sz = mean region size. WM-SAR’s cross-seed CV = 10.6% matches whole-graph methods (Full-Graph: 8.3%); all other compact-region methods reach CV = 39–173%.

Method	Rec-E $\uparrow$	Rec-T $\uparrow$	Rec-H $\uparrow$	Tok $\downarrow$	Sz $\downarrow$
WM-SAR (ours)	.500	.500	.987	855	8.3
Full-Graph-LLM	.240	.247	.953	2157	25.7
LLMRepair-Full-Plan-LLM	.180	.180	.980	2299	25.7
LocalRepair-2Hop-LLM	.167	.173	.953	1596	18.7
TraceScan-Full-LLM	.167	.180	.933	1284	25.7
TopK-5-LLM	.107	.127	.893	609	5.0
Window-8-LLM	.087	.107	.887	735	8.0
Window-4-LLM	.060	.080	.813	515	4.0
TraceScan-w2-LLM	.020	.080	.760	397	2.0
TraceScan-w1-LLM	.007	.033	.700	354	1.0
Greedy-Point-LLM	.007	.033	.700	363	1.0

Table 5: 3-LLM cross-model study ($n = 30$ per model, seed=42). McNemar exact p vs. LocalRepair-2Hop (Bonferroni $\alpha = 0.0167$): Gemini $p = 0.007$ (sig.); GPT-4o-mini $p = 0.065$ (marginal); GPT-4o $p = 0.42$ (not significant). Against Greedy/TopK-5/Window-4: all $p \leq 0.012$.

Method	GPT-4o-mini	GPT-4o	Gemini-2.5-Flash
WM-SAR (ours)	0.500	0.433	0.759
LocalRepair-2Hop	0.267	0.300	0.379
TopK-5	0.100	0.133	0.111
Window-4	0.000	0.000	0.000
Greedy-Point	0.000	0.000	0.000
Gap vs. LR-2Hop	+0.23	+0.13	+0.38
McNemar p (vs LR-2H)	0.065	0.42	0.007*

*sig. after Bonferroni ($\alpha = 0.0167$ for 3 models)

Table 6: Who&When attribution benchmark ($n = 30$, GPT-4o-mini). Rec-Exact = GT mistake step in selected region; Rec-Agent = any message by GT mistake agent in region; Size = mean region size. WM-SAR underperforms LocalRepair-2Hop on step-level attribution due to absent per-node error signals.

Method	Rec-Exact $\uparrow$	Rec-Agent $\uparrow$	Size $\downarrow$
LocalRepair-2Hop	0.60	0.63	4.9
TraceScan-w4-Point	0.37	0.70	4.0
WM-SAR	0.20	0.47	2.1
LastError-Point	0.00	0.30	1.0

subgraph that changes the residual dynamics.

8 Discussion

The root cause of engineering heuristic failure.

In SWE-bench, the relevant cascade is CODEANALYZER to PATCHWRITER to TESTRUNNER. Greedy-Point repairs TESTRUNNER (highest error) while CODEANALYZER re-propagates on the next step; LocalRepair-2Hop misses CODEANALYZER (three edges away). Repairing a bottleneck node does not remove the bottleneck *path* (Topping

et al., 2022). WM-SAR’s $\Delta\rho_{\text{rel}}$ criterion identifies the CA–PW–TR chain as the path whose removal most reduces $\rho(\mathbf{B}_{G_f \setminus R})$, regardless of which node shows the highest error.

Why Oracle coverage is not the same as spectral repair.

Table 1 shows that the Oracle and WM-SAR optimize different notions of repair. The Oracle covers the injected corrupted region, which is ideal for measuring attribution. WM-SAR instead removes the residual amplifier, which is ideal for preventing the next rollout from recreating the failure. The two objectives are complementary: coverage explains where the mistake happened, while spectral relief explains whether the repaired system will stay stable.

Applicability boundary — Who&When attribution.

On the Who&When blackbox attribution benchmark (Zhang et al., 2025), WM-SAR underperforms LocalRepair-2Hop on step-level attribution. WM-SAR requires observable per-node error $e(v)$ to compute GEAF; in blackbox attribution only the final answer is visibly wrong, so GEAF degenerates to the sink neighbourhood. It still degrades gracefully at the agent level, but its main advantage is scoped to settings exposing intermediate state errors, as our synthetic testbed does; Section 7.13 reports this boundary case.

Scalability. LocalRepair-3Hop’s advantage is partly an artefact of small graph size: the neighbourhood nearly becomes whole-graph repair. On larger traces, fixed-hop expansion grows less predictably, while WM-SAR can approximate growing-phase spectral radii with power iteration and retain the same residual objective.

Table 7: Full baseline comparison ($n = 200$, seed=42). TraceScan, Top-B, k-hop, PageRank, uncertainty, and full-plan LLM repair instantiate common engineering correctors: trace scanning, attribution ranking, graph neighbourhood repair, global influence scoring, uncertainty triage, and whole-context LLM repair. WM-SAR matches the strongest full-plan baselines on recovery while using $7.4\times$ fewer tokens per successful recovery than LLMRepair-Full-Plan.

Method	Rec $\uparrow$	Tokens	Tok/Rec $\downarrow$	PD-red $\uparrow$	DE-red% $\uparrow$
<i>WM-SAR and upper bounds</i>					
WM-SAR (ours)	.625	1,424	1,389	5.4	60.0
Oracle-Region	.625	475	402	5.8	60.1
LLMRepair-Full-Plan	.625	10,857	10,333	5.8	62.6
<i>Context-limited pointwise scanners</i>					
TraceScan-Full-Point	.620	21,714	20,623	0.8	47.0
Top-B-Nodes (K=3)	.620	600	600	0.8	47.0
TraceScan-w4-Point	.370	4,800	4,800	0.0	34.4
Top-B-Edges (K=3)	.320	600	600	0.0	28.7
PageRank-Subgraph	.140	518	532	0.4	23.2
k-hop-k2	.135	550	570	0.0	26.0
Uncertainty-Subgraph	.045	317	328	0.0	6.4
TraceScan-w2-Point	.000	1,200	—	0.0	16.4
TraceScan-w1-Point	.000	275	—	0.0	7.1
LastError-Point	.000	50	—	0.0	6.9
FirstFailedCall-Point	.000	50	—	0.0	0.0

Prompt structure, region focus, and stability.

Prompt formatting helps, but it is not the main effect. The larger gain comes from deciding what the LLM should see: WM-SAR removes distracting downstream symptoms while preserving the causal bridge. This is why its compact prompt matches the stability of whole-graph prompting without inheriting the noise of the full trace; Section 7.11 reports the paired tests and cross-seed variation.

9 Conclusion

Engineering heuristics optimize *observed* error but can leave the node-edge coupling operator $\mathbf{B}$ nearly unchanged, so errors keep compounding. The planning-regret bound identifies the repair objective that matters for the next rollout: minimize $\rho(\mathbf{B}_{G_f \setminus R})$. WM-SAR implements this objective with GEAF $\times\kappa$ seeding and $\Delta\rho$ -relief growing, producing a compact causal context for LLM repair. The takeaway is to ask “which connected subgraph, when repaired, most reduces future error growth?” not “which node has the most error?” Future work should evaluate real WebArena/ToolBench traces and learn the residual spectral predictor directly from trained world-model rollouts.

References

- Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Mikhail Burtsev, and Evgeny Burnaev. 2025. AriGraph: Learning knowledge graph world models with episodic memory for llm agents. In *International Joint Conference on Artificial Intelligence*.
- Kavosh Asadi, Dipendra Misra, and Michael Littman. 2018. Lipschitz continuity in model-based reinforcement learning. In *Proceedings of ICML*.
- Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, and 1 others. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*.
- Hyungjoo Chae, Namyoun Kim, Kai Tzu-iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. 2025. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. 2018. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*.
- Fan RK Chung. 1997. *Spectral Graph Theory*. American Mathematical Society.

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Tao Feng, Yexin Wu, Guanyu Lin, and Jiaxuan You. 2025. Graph world model. In *International Conference on Machine Learning*.
- Justin Gilmer, Kristof T Schütt, Anuroop Sriram, Matthew Reif, and 1 others. 2017. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*.
- Adam Gleave and Geoffrey Irving. 2022. Failure modes of learning reward models for LLMs and other sequence models. *arXiv preprint arXiv:2210.01790*.
- Google Cloud. 2026. Gemini 2.5 Flash. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/2-5-flash>.
- David Ha and Jürgen Schmidhuber. 2018. World models. *arXiv preprint arXiv:1803.10122*.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. 2019. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. 2023. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. 2019. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hanglei Ding, and 1 others. 2024b. AgentBench: Evaluating llms as agents. In *International Conference on Learning Representations*.
- Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 664–676.
- Jiaying Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyuan Hu, Yuchen Peng, and Wei Chen. 2025. AgentLens: Visual analysis for agent behaviors in llm-based autonomous systems. *IEEE Transactions on Visualization and Computer Graphics*, 31(8):4182–4197.
- Chang Ma, Junlei Zhang, Zhihao Liu, Croix Liu, Rui Ma, Yi Chen, and 1 others. 2024. AgentBoard: An analytical evaluation board of multi-turn llm agents. *arXiv preprint arXiv:2401.13178*.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36.
- Chaoxi Niu, Guansong Pang, and Ling Chen. 2023. Graph-level anomaly detection via hierarchical memory networks. In *Proceedings of ECML-PKDD*.
- OpenAI. 2024a. GPT-4o mini: Advancing cost-efficient intelligence. OpenAI blog.
- OpenAI. 2024b. GPT-4o system card. *arXiv preprint arXiv:2410.21276*.
- OpenAI. 2024c. Structured model outputs. <https://developers.openai.com/api/docs/guides/structured-outputs>.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37.
- Shuofei Qiao, Runnan Fang, Ningyu Zhang, Yuqi Zhu, Xiang Chen, Shumin Deng, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. Agent planning with world knowledge model. In *Advances in Neural Information Processing Systems*, volume 37.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. 2020. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Zhang, Weiming Lu, and Yueting Zhuang. 2023. TaskBench: Benchmarking large language models for task automation. In *Advances in Neural Information Processing Systems*, volume 36.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.
- Yifan Song, Weimin Xiong, Dawei Zhu, Li Cheng, Ke Wang, Tian Ye, and Sujian Li. 2023. Rest-GPT: Connecting large language models with real-world applications via RESTful APIs. *arXiv preprint arXiv:2306.06624*.
- Richard S Sutton. 1991. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163.
- Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. 2022. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.
- Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yue Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, and 1 others. 2024. Retroformer: Retrospective large language agents with policy gradient optimization. In *International Conference on Learning Representations*.
- Lunjun Zhang, Ge Yang, and Bradly C. Stadie. 2021. World model as a graph: Learning latent landmarks for planning. In *International Conference on Machine Learning*.
- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In *Proceedings of the International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 76583–76599.
- Andy Zhou, Kai Yan, Max Shlapentokh-Rothman, Hao-han Wang, and Yu-Xiong Wang. 2024a. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and 1 others. 2024b. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

Limitations

Synthetic evaluation. All experiments use 50 synthetic agent calling-tree graphs (≈ 26 nodes each) whose cascade dynamics ($e_{\text{child}} = 1.1 e_{\text{parent}} + \epsilon$) are generated programmatically rather than collected from real system traces. The benchmark-topology simulations in Section 7.7 match the reported structural statistics of SWE-bench, WebArena, and AgentBench-OS but do not consume actual failed rollout logs.

No real attributed failure traces. We do not evaluate on the multi-agent failure attribution datasets of Zhang et al. (2025), which provide ground-truth causal labels. Such datasets would allow direct validation of WM-SAR’s region selection against human-attributed root causes; we leave this to future work.

No parametric GWM training. The graph world model (GWM) error operators (L_X, M_X, L_A, M_A) are estimated from graph statistics rather than from a trained GCN/MPNN/GPS model. Evaluating WM-SAR

on rollouts from a trained parametric GWM (e.g., Error-Aware GWM, ActionNode-GWM) is an important validation step deferred to future work.

Repair operators. The broader repair design space includes state correction, edge-structure correction, re-rollout from a region boundary, target-cone rollback, and validator-boundary insertion. Our experiments implement only state correction (zeroing selected node errors) plus LLM root-cause identification (Phase 2). Edge-structure correction requires per-edge probability tracking absent from our graph schema, and target-cone rollback is out of scope at the current evaluation scale. Re-rollout from a boundary and validator-boundary insertion are **infeasible** without a live world model: they require re-simulating the rollout from the repair boundary or inserting a runtime validator API, neither of which exists in our failure-graph-only setting; they represent desiderata for real-WM deployment.

Objective-metric boundary. The repair objective $\rho(\mathbf{B}_{G_f \setminus R})$ is one of multiple valid repair criteria. The Who&When multi-agent failure attribution benchmark in Section 7.13 shows that GT-coverage-based methods outperform WM-SAR when ground-truth error signals are unavailable (Rec-Exact: LocalRepair-2Hop 0.60 vs. WM-SAR 0.20). WM-SAR requires per-node error magnitudes $e(v)$ to compute GEAF; in blackbox failure attribution settings (only final output visible), GEAF degenerates and spectral seeding loses its advantage.

Uncertainty term. The uncertainty score $\text{Unc}(v)$ in the theoretical WM-SAR formulation is computed but has no effect on outcomes at the default hyperparameters on the synthetic dataset (see the —Uncertainty row in Table 2). A dataset with meaningful uncertainty signals (e.g., ensemble disagreement in a trained parametric GWM) is needed to validate this component.

Ethics Statement

This work presents a graph-based algorithm for repairing failed AI agent rollouts. All experiments use synthetic simulation data generated programmatically; no human subjects, personal data, or sensitive information are involved. The LLM experiments use the OpenAI and Google Gemini APIs under standard research usage agreements (OpenAI,

2024a,b; Google Cloud, 2026). We foresee no direct dual-use risk in the repair algorithms presented. A potential concern is deployment in safety-critical agent pipelines (medical or legal decision-making) without sufficient validation on real traces; our Limitations section explicitly cautions against this.

A Appendix Overview

The appendix contains only material that would interrupt the main argument: proofs, hyperparameter calibration, dataset statistics, and prompt serialization details. All experimental evidence, including robustness checks and full baseline tables, is reported in the main text.

B Proofs of Theoretical Results

Proof of Theorem 1. Let $a_k = e_k^X$. Assumption 1 gives the scalar recursion

$$a_{k+1} \leq L_X a_k + \epsilon_X, \quad (15)$$

with $L_X \geq 0$ and $\epsilon_X \geq 0$. We claim that for every $k \geq 0$,

$$a_k \leq L_X^k a_0 + \epsilon_X \sum_{i=0}^{k-1} L_X^i, \quad (16)$$

where the empty sum is zero. The claim holds at $k = 0$. If it holds at k , then

$$\begin{aligned} a_{k+1} &\leq L_X a_k + \epsilon_X \\ &\leq L_X^{k+1} a_0 + \epsilon_X \sum_{i=1}^k L_X^i + \epsilon_X \\ &= L_X^{k+1} a_0 + \epsilon_X \sum_{i=0}^k L_X^i, \end{aligned} \quad (17)$$

which proves the induction. If $L_X \neq 1$,

$$\sum_{i=0}^{k-1} L_X^i = \frac{L_X^k - 1}{L_X - 1}; \quad (18)$$

if $L_X = 1$, the sum equals k . Substituting these two closed forms into Equation 16 gives Equation 2 and its $L_X = 1$ case. $\square$

Proof of Theorem 2. Write

$$\mathbf{B} = \begin{pmatrix} L_X & L_A \\ M_X & M_A \end{pmatrix}, \quad L_X, L_A, M_X, M_A \geq 0. \quad (19)$$

The characteristic polynomial is

$$\begin{aligned} p(\lambda) &= \lambda^2 - (L_X + M_A)\lambda \\ &\quad + (L_X M_A - L_A M_X). \end{aligned} \quad (20)$$

Thus the two eigenvalues are

$$\lambda_{\pm} = \frac{L_X + M_A \pm \sqrt{(L_X - M_A)^2 + 4L_A M_X}}{2}. \quad (21)$$

Since $\mathbf{B}$ is entrywise nonnegative, the Perron–Frobenius theorem implies that its spectral radius is a real nonnegative eigenvalue and equals the largest real root. Therefore $\rho(\mathbf{B}) = \lambda_+$, which is Equation 4.

It remains to prove strict super-additivity. If $L_A M_X > 0$, then

$$\sqrt{(L_X - M_A)^2 + 4L_A M_X} > |L_X - M_A|. \quad (22)$$

When $L_X \geq M_A$, Equation 21 gives

$$\rho(\mathbf{B}) > \frac{L_X + M_A + (L_X - M_A)}{2} = L_X. \quad (23)$$

When $M_A \geq L_X$, the same argument gives

$$\rho(\mathbf{B}) > \frac{L_X + M_A + (M_A - L_X)}{2} = M_A. \quad (24)$$

Hence $\rho(\mathbf{B}) > \max(L_X, M_A)$ whenever the two channels are coupled in both directions. $\square$

Proof of Theorem 3. Let $J(\pi)$ denote the discounted value of policy π under the true rollout and $\hat{J}(\pi)$ the value computed by the learned graph world model. For a fixed π , Assumption 3 gives

$$\begin{aligned} |J(\pi) - \hat{J}(\pi)| &\leq L_R \kappa \sum_{t=0}^{H-1} \gamma^t \|z_t\| + \epsilon_R H \\ &\leq L_R \kappa \epsilon \sum_{t=0}^{H-1} (\gamma \rho(\mathbf{B}))^t + \epsilon_R H \\ &= L_R \kappa \epsilon \Phi_H(\gamma, \rho(\mathbf{B})) + \epsilon_R H. \end{aligned} \quad (25)$$

Define

$$C_H = L_R \kappa \epsilon \Phi_H(\gamma, \rho(\mathbf{B})) + \epsilon_R H. \quad (27)$$

Equation 26 holds uniformly for both π^* and $\hat{\pi}$. Since $\hat{\pi} \in \arg \max_{\pi} \hat{J}(\pi)$,

$$\hat{J}(\pi^*) - \hat{J}(\hat{\pi}) \leq 0. \quad (28)$$

Using this optimality inequality,

$$\begin{aligned} J(\pi^*) - J(\hat{\pi}) &= [J(\pi^*) - \hat{J}(\pi^*)] \\ &\quad + [\hat{J}(\pi^*) - \hat{J}(\hat{\pi})] \\ &\quad + [\hat{J}(\hat{\pi}) - J(\hat{\pi})] \\ &\leq |J(\pi^*) - \hat{J}(\pi^*)| \\ &\quad + |\hat{J}(\hat{\pi}) - J(\hat{\pi})| \\ &\leq 2C_H. \end{aligned} \quad (29)$$

Substituting the definition of C_H yields Equation 5. $\square$

C Hyperparameters and Implementation

Parameter	Value
H (spectral walk depth)	4
w (weight norm proxy)	0.9
$K_{\max}$ (max region size)	20
k_0 (initial seeds)	6
λ_1 (error weight in grow)	1.2
λ_2 (ρ -relief weight in grow)	1.5
λ_3 (cost penalty)	0.1
Merge Jaccard threshold τ	0.5
γ (planning discount)	0.95
Cascade gain α	1.1
Cascade noise σ	0.05

L_X, L_A, M_X, M_A **estimation.** In the failure graph, GWM weights are unavailable. We estimate: $L_X = w \cdot \rho(A_R)$; $L_A = w \cdot \beta_A d_{\text{in}} \bar{e}$ with $\beta_A = 0.3$; $M_X = w \cdot \beta_X d_{\text{out}} \bar{e}$ with $\beta_X = 0.2$; $M_A = w \cdot \beta_M f_{\text{high}}$ with $\beta_M = 0.5$, where $d_{\text{in/out}}$ = mean edge-type diversity, $\bar{e}$ = mean error, f_{high} = fraction of edges between high-error nodes.

Calibration of constants (0.3, 0.2, 0.5). The constants in L_A (0.3), M_X (0.2), and M_A (0.5) are *calibration factors* rather than theoretically derived values. Their purpose is to ensure that $\rho(\mathbf{B})$ estimated from graph statistics is of the same order of magnitude as the local spectral quantity $\rho(A)$, so that the combined score $e(v) \cdot \widehat{\text{GEAF}}_v \cdot (1 + \kappa_v)$ balances all three contributions comparably. They are **not** derived from or constrained by Theorem 2; that theorem requires only $L_A M_X > 0$ (which holds for any positive constants) to guarantee super-additivity. The mean super-additivity ratio $\rho(\mathbf{B}) / \max(L_X, M_A) - 1 = 0.007$ confirms that the resulting coupling strength is non-negligible, not merely formally positive. In a setting with a trained parametric GWM, these constants can be replaced by direct Lipschitz estimates from the weight matrices.

D Dataset Statistics

Property	Value
Instances	50 (seed=42)
N (nodes)	22–30 (mean 25.7)
Node / edge types	9 / 6
State feature dim	8
Failure types	4 (drift, misfire, cascade, validator)
Root cause	planner (30%) / executor (70%)
Pre-repair $\rho(\mathbf{B})$	1.97 ± 0.30
Coupled-operator super-additivity	100%
GrowthSlope (unre-paired)	+0.0026

E LLM Prompt and Serialization Details

The main method only requires the LLM to see the selected repair region, not the full planning graph. We serialize nodes in topological order and include node type, local state features, observed error, confidence, and boundary edges. Edges crossing into or out of the selected region are marked as external context, so the model can understand the role of the region without seeing the whole graph.

Structured prompt. The system prompt tells the LLM that it is a failure analyst for a multi-agent calling tree and asks it to identify the node that introduced the initial mistake. The user prompt contains the serialized subgraph and requests a JSON object with root-cause node identifiers and a short rationale. JSON output is used only to make evaluation deterministic; the repair advantage comes from the selected context, not from a special decoding trick (OpenAI, 2024c).

Repair operator. In the simulation, the returned repair is applied by zeroing the selected node errors and re-simulating the rollout. In a deployed world-model system, the same decision would trigger a targeted re-run of the selected planner, executor, validator, tool call, or memory update.